Institución Universitaria

Facultad de Ciencias Exactas y Aplicadas
Física Computacional 1
Robinson Longas Bedoya
robinsonlongas3586@correo.itm.edu.co
robinsonlongas@itm.edu.co

- **Solución de ecuaciones no lineales y trascendentes**
- **Raíces de polinomios**

Bibliografía

- *Computational Physics*, Mark Newman.
- *Programming for computations - Python*, Evein Linge and Petter Langtangen.
- *Computational Physics with Python*, Eric Ayars.
- *Computational mathematics: An introduction to Numerical Analysis and Scientific Computing with Python*, Dimitrios Mitsotakis.
- *Computational physics: problem solving with Python*, Rubin H. Landau, Manuel J. Páez and Cristian C. Bordeianu.

Solución de ecuaciones no lineales y trascendentes

Suponga que se lanza un cuerpo con una velocidad inicial inicial $\vec{v}_0$ formando un ángulo θ con la horizontal. En Física básica se muestra que las ecuaciones de movimiento para este sistema son

$$x(t) = x_0 + v_0 \cos\theta t\,,$$
$$y(t) = y_0 + v_0 \sin\theta t - \frac{g}{2}t^2\,.$$

Si queremos saber el tiempo en que el cuerpo cae al piso de nuevo, debemos resolver la ecuación $y(T) = 0$. Cuya solución puede escribirse como

$$T = \frac{v_0 \sin\theta + \sqrt{(v_0 \sin\theta)^2 + 2gy_0}}{g}\,.$$

Con ese tiempo, podemos escribir el alcance horizontal del proyectil como

$$x(\theta) = x_0 + \frac{v_0 \cos\theta}{g}\left[v_0 \sin\theta + \sqrt{(v_0 \sin\theta)^2 + 2gy_0}\right] \tag{1}$$

La ecuación (1) es una ecuación no lineal en θ. Si de entrada conocemos el ángulo de lanzamiento, simplemente lo reemplazamos en dicha ecuación. Pero, en la mayoría de casos, podemos medir el alcance horizontal y queremos determinar el ángulo de lanzamiento. Es decir, dado x, queremos saber θ. Su solución requiere de métodos numéricos.

La forma más simple de hacerlo es graficar el lado izquierdo y el lado derecho para mirar en donde se cruzan. Por otro lado, existen métodos para hacerlo: *bisección, Newton, secante*. En nuestro curso no vamos a estudiar con detalle estos métodos, vamos a utilizar las librerías científicas de Python.

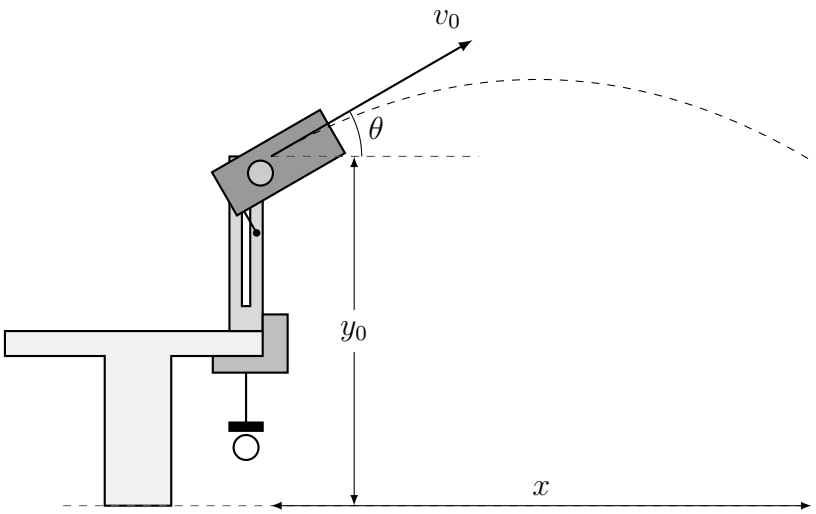


Figura 1: Montaje de laboratorio para lanzamiento de proyectiles con altura inicial y_0.

raices_parabolico.py

```
""" En este programa se utiliza la librería scipy
    para encontrar los ángulos de lanzamiento de un cuerpo para
    una altura máxima y una distancia horizontal máxima"""

import scipy as sp
import numpy as np
import matplotlib.pyplot as plt

def x_max(theta, x0=0., y0 = 0., v0 = 3.5, g = 9.8):
    """ función para encontrar el desplazamiento máximo horizontal.
    Variables de entrada. theta:  ángulo en metros. y0: altura inicial en
   metros.
    v0: magnitud de la velocidad inicial en m/s. g: aceleración gravedad en
   m/s^2"""
    sintheta = np.sin(np.radians(theta))
    costheta = np.cos(np.radians(theta))
    prefactor = ( v0 * costheta )/g
    return x0 + prefactor * (v0 * sintheta + np.sqrt(v0**2 * sintheta**2 +
   2*g*y0) )

def y_max(theta, y0 = 0., v0 = 3.5, g = 9.8):
    """ función para encontrar la altura máxima.
    Variables de entrada. theta:  ángulo en metros. y0: altura inicial en
   metros.
    v0: magnitud de la velocidad inicial en m/s. g: aceleración gravedad en
   m/s^2"""
    sintheta = np.sin(np.radians(theta))
    return y0 + 1/(2.*g) * v0**2 * sintheta**2

def f_x(theta, xmaximo, x0, y0, v0, g):
    """Función asociada al alcance horizontal máximo:  xmaximo - x_max(theta
```

```
    ,...) = 0"""
27      return x_max(theta, x0, y0, v0, g) - xmaximo
28
29  def f_y(theta, ymaxima, y0, v0, g):
30      """Función asociada a la altura máxima:  ymaximo - y_max(theta,...) = 0"
        ""
31      return y_max(theta, y0, v0, g) - ymaxima
32
33
34  if __name__ == "__main__":
35      y0 = 1.5       # Altura inicial (m)
36      x0 = 0.0       # Posición inicial (m)
37      v0 = 35.0      # Velocidad inicial (m/s)
38      g = 9.8        # Gravedad (m/s^2)
39      xmaximo = 89.0 # Alcance máximo (m)
40
41      angulos = np.linspace(0, 90, 300)
42      alcancesx = x_max(angulos, x0, y0, v0, g)
43
44      plt.figure(figsize=(8, 5))
45      plt.plot(angulos, alcancesx, label=r'$x_{\mathrm{max}}(\theta)$', color=
        'blue', lw=2)
46      plt.axhline(xmaximo, color='red', linestyle='--', label=rf'$x_{{max}} =
        $ {xmaximo} m')
47      plt.title(f'Alcance Horizontal vs Ángulo ($v_0 = {v0}$ m/s, $y_0 = {y0}$
        m)')
48      plt.xlabel(r'$\theta$ (grados)')
49      plt.ylabel('$x$ (m)')
50      plt.grid(True, linestyle=':', alpha=0.7)
51      plt.legend()
52      plt.tight_layout()
53
54      try:
55          sol1 = sp.optimize.root_scalar(f_x, args=(xmaximo, x0, y0, v0, g),
        bracket=[0, 45], method='brentq')
56          sol2 = sp.optimize.root_scalar(f_x, args=(xmaximo, x0, y0, v0, g),
        bracket=[45, 90], method='brentq')
57
58          res1 = f_x(sol1.root, xmaximo, x0, y0, v0, g)
59          res2 = f_x(sol2.root, xmaximo, x0, y0, v0, g)
60
61          print(f"\n--- Ángulos con que se debe lanzar la pelota para un
        alcance máximo de = {xmaximo} metros---")
62          print(f"Ángulo 1  : {sol1.root:.4f}°| Error: {abs(res1):.2e} m  |
        Iteraciones: {sol1.iterations}")
63          print(f"Ángulo 2 : {sol2.root:.4f}° | Error: {abs(res2):.2e} m |
        Iteraciones: {sol2.iterations}")
64
65      except ValueError:
66          # Ocurre un error si no encuentra soluciones
```

```
67          alcance_max_posible = np.max(alcancesx)
68          print(f"\n[Error] El alcance deseado de {xmaximo} m es físicamente
    inalcanzable.")
69          print(f"El alcance máximo posible con v0={v0} m/s y y0={y0} m es de
    {alcance_max_posible:.2f} m.")
70
71      #plt.savefig('../../figuras/alcance_vs_angulo.pdf', bbox_inches='tight')
72      plt.show()
```

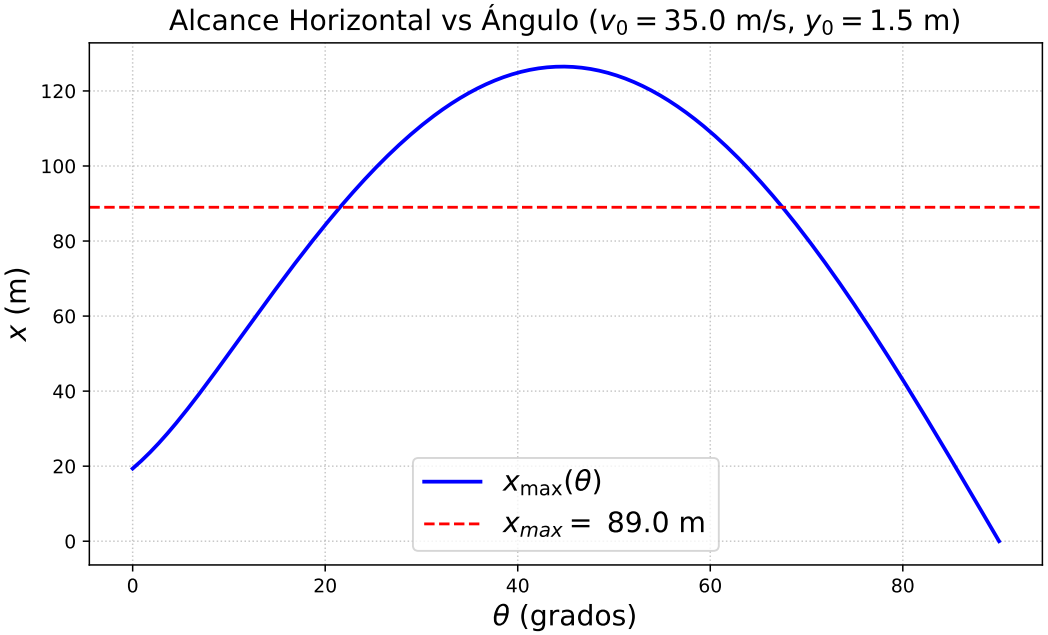


Figura 2: Alcance máximo en función del ángulo de lanzamiento.

Ejemplo:

Encontremos la solución de la ecuación $x = -\ln(x) \implies f(x) = x + \ln(x) = 0$ de manera numérica.

La función W de Lambert

En matemáticas se define la función de Lambert W como

$$z = W(z)e^{W(z)}$$

Noten pues que en nuestro caso, con

$$x + \ln x = 0 \implies x = -\ln x \implies e^x = e^{-\ln x} = e^{\ln(x)^{-1}} = \frac{1}{x} \implies xe^x = 1$$

Si comparamos con función W podemos ver que la solución a la ecuación $ue^u = y$ es $u = W(y)$. Luego, la solución a la ecuación $xe^x = 1$ es $W(1)$

raices_xlnx.py

```python
import scipy as sp
import numpy as np
import matplotlib.pyplot as plt

def f(x):
    return x + np.log(x)

def df(x):
    return 1 + 1/x

x = np.linspace(0.01, 10, 100)
fx = f(x)

plt.figure(figsize=(8, 5))
plt.plot(x, x, color='blue', lw=2, label=rf'$f(x) = x$' )
plt.plot(x, -np.log(x), color='red', linestyle='--', label=rf'$f(x)= - \ln({{x}})  $',lw=2)
plt.ylabel(r'$f(x)$',fontsize=15)
plt.xlabel(r'$x$ ',fontsize=15)
plt.grid(True, linestyle=':', alpha=0.7)
plt.legend(fontsize=15)
plt.tight_layout()

sol_brent = sp.optimize.root_scalar(f, bracket=[0.1, 1.0], method='brentq')
sol_secante = sp.optimize.root_scalar(f, x0=1.0, method='secant')
sol_bisect  = sp.optimize.root_scalar(f, bracket=[0.1, 1.0], method='bisect',xtol=1e-8)
sol_newton  = sp.optimize.root_scalar(f, x0=1.0, fprime=df, method='newton')
sol_lambert = np.real(sp.special.lambertw(1))

metodos = [
    ("Brent (brentq)", sol_brent.root, sol_brent.iterations, sol_brent.function_calls),
    ("Bisección", sol_bisect.root, sol_bisect.iterations, sol_bisect.function_calls),
    ("Secante", sol_secante.root, sol_secante.iterations, sol_secante.function_calls),
    ("Newton", sol_newton.root, sol_newton.iterations, sol_newton.function_calls),
    ("Lambert W", sol_lambert, 1, 1)
]

print(f"{'Método':<18} | {'Raíz Hallada':<14} | {'Residuo f(x)':<14} | {'Iteraciones':<12} | {'Llamadas f(x)':<14}")
print("-" * 80)
for nombre, raiz, iters, nfev in metodos:
    residuo = abs(f(raiz))
```

```
    print(f"{nombre:<18} | {raiz:<14.8f} | {residuo:<14.2e} | {iters:<12} | {nfev:<14}")

#plt.savefig('../../figuras/xlnx.pdf', bbox_inches='tight')
plt.show()
```

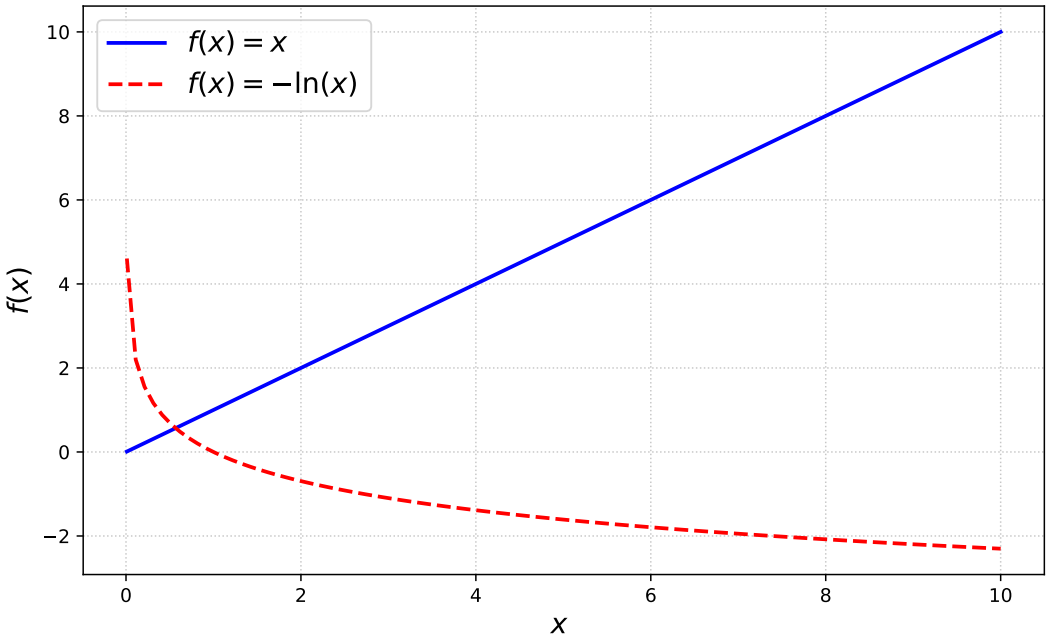


Figura 3: Gráfica de x y $-\ln(x)$ vs x.

Resumen de `root_scalar`

♦ Acceda a la documentación oficial `scipy.optimize.root_scalar` dando clic aquí.

```
sol=sp.optimize.root_scalar(f, args=(), method=None, bracket=None, fprime=None,
                            fprime2=None, x0=None, x1=None, xtol=None, rtol=None,
                            maxiter=None, options=None)
```

`root_scalar(f, bracket=[a,b], method='brentq')`	(Método de Brent)
`root_scalar(f, bracket=[a,b], method='bisect')`	(Método de Bisección)
`root_scalar(f, x0=x0, x1=x1, method='secant')`	(Método de la Secante)
`root_scalar(f, x0=x0, fprime=df, method='newton')`	(Método de Newton-Raphson)

1. Parámetros Generales

- **`f`:** La función escalar $f(x)$ cuya raíz se desea encontrar.
- **`args`:** Tupla con argumentos adicionales para $f(x, a, b)$ si la función los requiere (`args=(a, b)`).

- **`xtol` / `rtol`:** Tolerancia absoluta y relativa esperada para el error en x.
- **`maxiter`:** Número máximo de iteraciones permitidas antes de detener el algoritmo.
- **`options`:** Diccionario de opciones adicionales específicas del método (p. ej., `{'disp': True}`).

2. Métodos

- **Método de Brent (`method='brentq'`):** Algoritmo híbrido que combina bisección, secante e interpolación cuadrática inversa. Es el método estándar de propósito general para funciones continuas en un intervalo. Requiere obligatoriamente un cambio de signo en el intervalo ($f(a) \cdot f(b) < 0$). Ofrece convergencia superlineal con la garantía de estabilidad de la bisección.
  - **`bracket`:** Intervalo $[a, b]$ obligatorio donde $f(a) \cdot f(b) < 0$.
  - *Parámetros opcionales:* `args`, `xtol`, `rtol`, `maxiter`.
- **Método de Bisección (`method='bisect'`):** Divide reiteradamente el intervalo a la mitad y selecciona el subintervalo donde ocurre el cambio de signo.
  - **`bracket`:** Intervalo $[a, b]$ obligatorio con cambio de signo ($f(a) \cdot f(b) < 0$).
  - *Parámetros opcionales:* `args`, `xtol`, `rtol`, `maxiter`.
- **Método de la Secante (`method='secant'`):** Método abierto que aproxima la derivada $f'(x)$ mediante la pendiente de la recta secante formada por dos puntos. No requiere intervalo cerrado con cambio de signo ni derivada analítica. Se recomienda pasar dos semillas iniciales (`x0` y `x1`); si se omite `x1`, SciPy genera automáticamente un punto cercano $x_1 = x_0 + \delta$.
  - **`x0`:** Primer punto o semilla inicial (obligatorio).
  - **`x1`:** Segundo punto inicial (recomendado para definir el paso de la secante).
  - *Parámetros opcionales:* `args`, `xtol`, `rtol`, `maxiter`.
- **Método de Newton-Raphson (`method='newton'`):** Implementa la iteración $x_{k+1} = x_k - f(x_k)/f'(x_k)$ utilizando la tangente a la curva en el punto actual. Presenta convergencia cuadrática cerca de la raíz. Requiere pasar explícitamente la función derivada $f'(x)$ mediante el parámetro `fprime`.
  - **`x0`:** Punto o semilla inicial (obligatorio).
  - **`fprime`:** Función de Python que retorna la primera derivada $f'(x)$ (obligatorio).
  - *Parámetros opcionales:* `fprime2` (para Halley), `args`, `xtol`, `rtol`, `maxiter`.

3. Atributos del objeto de retorno (`sol`)

- **`sol.root`:** Valor numérico de la raíz encontrada.
- **`sol.converged`:** Booleano que indica si el algoritmo convergió con éxito (`True`/`False`).
- **`sol.iterations`:** Número de iteraciones ejecutadas.
- **`sol.function_calls`:** Número total de evaluaciones realizadas a la función $f(x)$.

Raíces de polinomios

El problema de encontrar las n raíces de un polinomio de grado n es una tarea computacionalmente relevante. Los métodos analíticos que nos enseñan tradicionalmente se vuelven imprácticos para $n > 3$.

Por ejemplo, encontrar las raíces de $p(x) = x^2 - 2x + 1$ es trivial, pero determinar las raíces de

$$p(x) = 3x^7 - 4x^5 + 6x^2 - x + 3$$

requiere de métodos numéricos. En Python disponemos de dos aproximaciones con NumPy: la función clásica `np.roots` y la moderna `np.polynomial.Polynomial`. Puede encontrar la documentación dando clic aquí.

raices_polinomios.py

```python
import numpy as np
import matplotlib.pyplot as plt

#======= Polinomio: 3x^7 - 4x^5 + 0x^4 + 0x^3 + 6x^2 - x + 3 = 0
    ==============

coeff = [3, -1, 6, 0, 0, -4, 0, 3] # note el orden, de menor a mayor
p = np.polynomial.Polynomial(coeff)
raices_p = p.roots()
print("Polinomio:")
print(p)
print("\nRaíces encontradas:")
for i, r in enumerate(raices_p):
    valorevaluado = p(r)
    print(f"  x_{i+1} = {r:.4f}")
    print(f"|residuo|: {abs(valorevaluado):.2e}")

# ===== plot ========
x = np.linspace(-2, 2, 400)
y = np.polyval(coeff, x)
plt.figure(figsize=(10, 7))
plt.plot(x, y, color='red', linestyle='-', label=rf'$p(x)= 3x^7 - 4x^5 + 6x
    ^2 - x + 3 $',lw=2)
plt.xlabel(r'$x$ ',fontsize=20)
plt.grid(True, linestyle=':', alpha=0.7)
```

```
plt.legend(fontsize=20)
plt.tick_params(axis='both', which='both', labelsize=17) # tamño de los ejes
plt.tight_layout()
plt.savefig('../../figuras/polinomio.pdf', bbox_inches='tight')

plt.show()
```

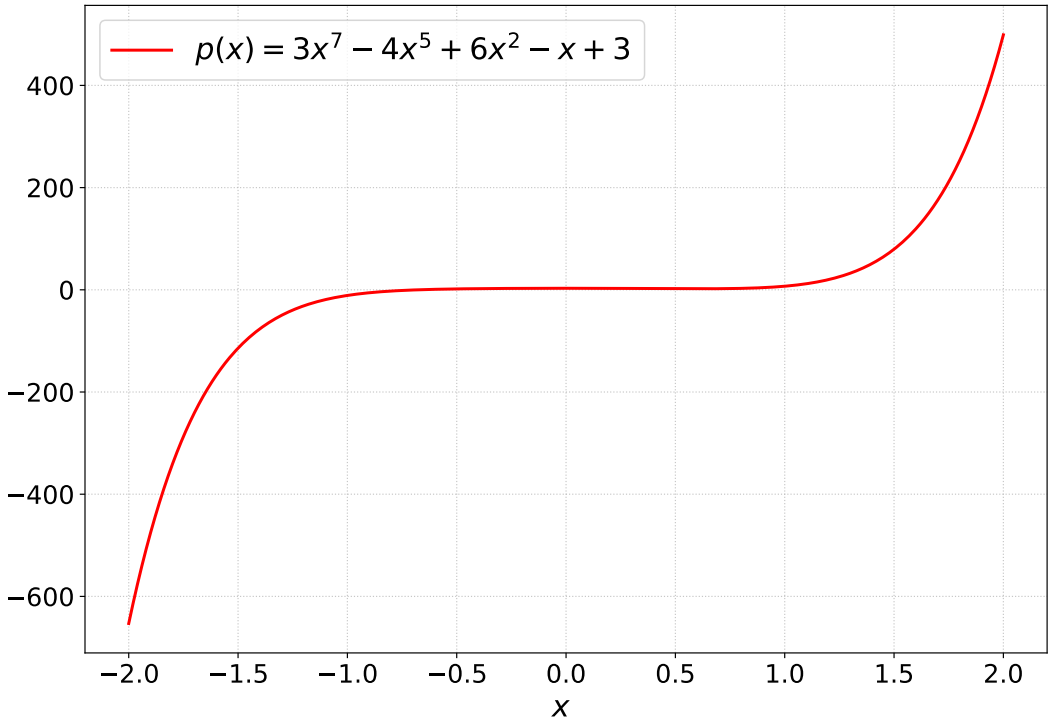


Figura 4: Gráfica de $p(x) = 3x^7 - 4x^5 + 6x^2 - x + 3$ vs x.

Resumen

♦ Acceda a la documentación de la API (Application Programming Interface) `numpy.polynomial` dando clic aquí.

`p = np.polynomial.Polynomial(coeff)` (Creación del objeto polinomio $P(x)$)
`raices_p = p.roots()` (Cálculo de todas las raíces complejas/reales)
`valorevaluado = p(r)` (Evaluación directa en un punto o arreglo x)

---

1. `coeff = [a_0, a_1, ..., a_n]`

La clase `Polynomial` requiere ingresar los coeficientes en orden ascendente de potencias ($a_0 + a_1x + a_2x^2 + \cdots + a_nx^n$). Es obligatorio incluir ceros para los grados ausentes. Por ejemplo, en $3x^7 - 4x^5 + 6x^2 - x + 3$, las potencias x^3, x^4, x^6 se representan explícitamente con 0.

2. `p = np.polynomial.Polynomial(coeff)`

- **Qué hace:** Crea un objeto de clase algebraica que encapsula las propiedades del polinomio.

- **Representación:** Al imprimir `print(p)`, genera automáticamente la expresión formateada $3{,}0 - 1{,}0\,x + 6{,}0\,x^2 - 4{,}0\,x^5 + 3{,}0\,x^7$. El símbolo pude cambiarse `p = np.polynomial.Polynomial(coeff,symbol='t')` para que imprima $3{,}0 - 1{,}0\,t + 6{,}0\,t^2 - 4{,}0\,t^5 + 3{,}0\,t^7$

3. `p.roots()`

- **Qué devuelve:** Un arreglo numérico con todas las raíces del polinomio (tanto reales como pares complejos conjugados).
- **Método interno:** Calcula los valores propios (autovalores) de la matriz compañera escalada para garantizar máxima estabilidad numérica.

4. Evaluaciones y Residuos: `p(r)`

- **Evaluación directa:** El objeto `p` puede ser llamado como una función estándar de Python (`p(x)`), soportando escalares o arreglos de NumPy vectorizados.
- **Cálculo del residuo:** Evaluar el polinomio en la raíz calculada $P(r)$ permite medir el error numérico en punto flotante ($|P(r)| \approx 0$).

Problemas:

1. Resuelva las siguientes ecuaciones utilizando el método gráfico y las rutinas de `scipy.optimize`

   a) $x = 2 - e^{-x}$

   b) $x^3 + 2x = e^{-x}$

   c) $x = \cos x$

2. La ley de radiación de Planck establece que la intensidad de la radiación por unidad de área y por unidad de longitud de onda λ de un cuerpo negro a una temperatura T es:

$$I(\lambda) = \frac{2\pi hc^2\lambda^{-5}}{e^{hc/\lambda k_B T} - 1},$$

   donde h es la constante de Planck, c es la velocidad de la luz y k_B es la constante de Boltzmann.

   a) Muestre, mediante derivación, que la longitud de onda λ en la cual la radiación emitida es máxima es la solución de la ecuación:

$$5e^{-hc/\lambda k_B T} + \frac{hc}{\lambda k_B T} - 5 = 0.$$

   Realice la sustitución $x = hc/\lambda k_B T$ y demuestre a partir de ello que la longitud de onda de radiación máxima obedece la *ley del desplazamiento de Wien*:

$$\lambda = \frac{b}{T},$$

   donde la llamada *constante de desplazamiento de Wien* es $b = hc/k_B x$, y x es la solución a la ecuación no lineal:

$$5e^{-x} + x - 5 = 0.$$

b) Escriba un programa para resolver esta ecuación con una precisión de $\epsilon = 10^{-6}$ utilizando el método de búsqueda binaria (bisección), y encuentre así un valor para la constante de desplazamiento.

c) La ley de desplazamiento es la base del método de *pirometría óptical*, un método para medir la temperatura de objetos mediante la observación del color de la radiación térmica que emiten. El método se utiliza comúnmente para estimar las temperaturas superficiales de cuerpos astronómicos, como el Sol. El pico de la longitud de onda en la radiación emitida por el Sol se encuentra en $\lambda = 502$ nm. A partir de las ecuaciones anteriores y su valor de la constante de desplazamiento, estime la temperatura superficial del Sol.

3. Un problema de ingeniería importante que surge en múltiples aplicaciones es la vibración de una viga empotrada en un extremo y libre en el otro. Este problema se puede analizar analíticamente, pero los cálculos se reducen a resolver la siguiente ecuación algebraica no lineal:

$$\cosh\beta\cos\beta = -1,$$

donde β se relaciona con parámetros clave de la viga a través de

$$\beta^4 = \omega^2\frac{\rho A}{EI},$$

siendo ρ la densidad de la viga, A el área de la sección transversal, E el módulo de Young e I el momento de inercia de la sección transversal. El parámetro de mayor interés es ω, que representa la frecuencia angular de la viga.

Deseamos calcular las frecuencias de una viga de acero en vibración con una sección transversal rectangular de ancho $b = 25$ mm y alto $h = 8$ mm. La densidad del acero es $\rho = 7850$ kg/m^3 y su módulo de Young es $E = 2 \times 10^{11}$ Pa. El momento de inercia de una sección transversal rectangular está dado por $I = bh^3/12$.

a) Grafique la ecuación a resolver para que se pueda inspeccionar dónde ocurren los cruces por cero.

Sugerencia: Al escribir la ecuación como $f(\beta) = 0$, la función f aumenta su amplitud de forma dramática a medida que crece β. Por lo tanto, es prudente analizar una ecuación con amplitud amortiguada, $g(\beta) = e^{-\beta}f(\beta) = 0$. Grafique la función g en su lugar.

b) Calcule las tres primeras frecuencias de la viga.

4. Considere el polinomio de grado seis

$$P(x) = 924x^6 - 2772x^5 + 3150x^4 - 1680x^3 + 420x^2 - 42x + 1.$$

a) Realice una gráfica de $P(x)$ en el intervalo $x \in [0, 1]$. A partir de la inspección visual de la gráfica, estimar los valores aproximados de las seis raíces del polinomio (los puntos donde la función se anula).

b) Utilizando las funciones integradas de álgebra lineal de NumPy (`np.roots` o `np.polynomial.Polynomial.roots`) escriba un programa en Python para encontrar las posiciones de las seis raíces y muestre sus residuos.

5. **El punto de Lagrange:** Existe un punto especial entre la Tierra y la Luna, conocido como el punto de Lagrange L_1, en el cual un satélite orbita alrededor de la Tierra en perfecta sincronía con la Luna, manteniéndose siempre entre ambos cuerpos. Esto ocurre porque la atracción gravitacional de la Tierra hacia adentro y la de la Luna hacia afuera se combinan para generar exactamente la fuerza centrípeta requerida para mantener al satélite en su órbita.

   a) Suponiendo órbitas circulares y que la masa de la Tierra es mucho mayor que la de la Luna o el satélite, demuestre que la distancia r desde el centro de la Tierra al punto L_1 satisface la ecuación de equilibrio de fuerzas:

   $$\frac{GM}{r^2} - \frac{Gm}{(R-r)^2} = \omega^2 r,$$

   donde R es la distancia Tierra-Luna, M y m son las masas de la Tierra y la Luna respectivamente, G es la constante de gravitación universal, y ω es la velocidad angular de la Luna y del satélite.

   b) Al multiplicar por $r^2(R-r)^2$ y reordenar términos, demuestre que la ecuación anterior se reduce a un polinomio de grado cinco en r:

   $$\omega^2 r^5 - 2\omega^2 R r^4 + \omega^2 R^2 r^3 + G(m-M)r^2 + 2GMRr - GMR^2 = 0.$$

   c) Escriba un programa en Python que resuelva la posición del punto L_1 utilizando la API de polinomios de NumPy (`np.polynomial.Polynomial.roots`). Seleccione la raíz físicamente válida ($0 < r < R$) e identifique las demás raíces complejas/no físicas. Los valores para los parámetros físicos son:

   $$G = 6{,}674 \times 10^{-11}\ \text{m}^3\text{kg}^{-1}\text{s}^{-2}, \qquad M = 5{,}974 \times 10^{24}\ \text{kg},$$
   $$m = 7{,}348 \times 10^{22}\ \text{kg}, \qquad R = 3{,}844 \times 10^{8}\ \text{m},$$
   $$\omega = 2{,}662 \times 10^{-6}\ \text{s}^{-1}.$$

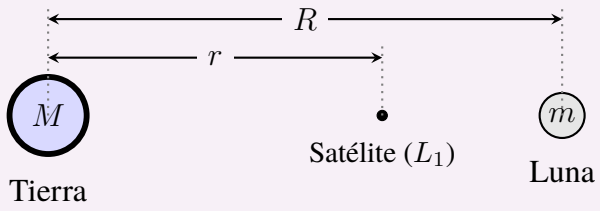


Figura 5: Esquema del sistema Tierra–Luna y la posición del punto L_1.